RAG-Based Customer Support Assistant

High-Level Design (HLD)

1. System Overview

This system is designed to answer user questions from a PDF document using Artificial Intelligence.

Instead of manually searching through documents, the system retrieves relevant information and generates accurate answers.

The system uses a Retrieval-Augmented Generation (RAG) approach, where data is first retrieved from a document and then used by a language model to generate responses.

- **Problem Definition**

  Users often find it difficult to search large documents manually.

  This system solves the problem by allowing users to ask questions and receive direct answers from the document.

- **Scope of the System**

  Input: PDF document and user query

  Output: Context-based answer

  The system is focused on customer support use cases, such as answering policy-related questions.

2. System Architecture Diagram

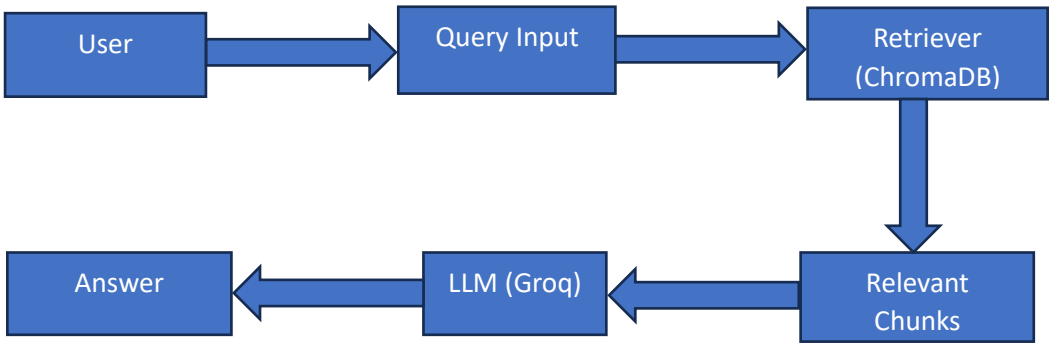

Figure: Architecture of RAG-Based Customer Support Assistant

- **Data Flow:**

  The system processes the PDF by splitting it into chunks, generating embeddings, and storing them in ChromaDB.

  When a user query is given, relevant chunks are retrieved and passed to the LLM to generate the final answer.

3. Component Description

This section describes the key components of the RAG-based Customer Support Assistant system.

Each component plays an important role in processing the document, retrieving relevant information,

and generating accurate responses to user queries. The system is designed using a modular approach,

where each component performs a specific function in the overall workflow.

- **Document Loader**

  The Document Loader is responsible for loading the PDF file into the system.

  In this project, PyPDFLoader is used to read the PDF and convert it into a structured format for processing.

- **Chunking Strategy**

  The chunking strategy divides the document into smaller parts to improve retrieval efficiency.

  RecursiveCharacterTextSplitter is used to split the document into chunks with overlap, ensuring better context retention.

- **Embedding Model**

  The embedding model converts text into numerical vectors.

  HuggingFaceEmbeddings (all-MiniLM-L6-v2) is used to generate embeddings for each chunk.

- **Vector Store**

  The vector store is used to store embeddings for efficient retrieval.

  ChromaDB is used as the vector database to store and manage embeddings.

- **Retriever**

  The retriever fetches relevant document chunks based on user queries.

It compares query embeddings with stored embeddings and returns the most relevant results.

- **LLM (Large Language Model)**

The LLM generates answers using the retrieved context.

In this system, ChatGroq (LLaMA model) is used to generate accurate responses.

- **Graph Workflow Engine (LangGraph)**

LangGraph is used to control the workflow of the system.

It defines nodes and edges to manage the flow of data between components.

- **Routing Layer**

The routing layer decides how queries are processed.

It determines whether the query should be answered directly or escalated.

- **HITL Module (Human-in-the-Loop)**

The HITL module handles escalation to human support.

If the system cannot generate a confident answer, the query is forwarded to a human.

4. Data Flow

The system processes the PDF by splitting it into chunks, generating embeddings, and storing them in ChromaDB.

When a user query is given, relevant chunks are retrieved using the retriever.

These chunks are then passed to the LLM, which generates the final answer.

- **Query Lifecycle**

1. User inputs a query.
2. The query is converted into embeddings.
3. The retriever searches for relevant chunks.
4. Relevant data is passed to the LLM.
5. The LLM generates the final response.
6. If the response is insufficient, it is escalated to human support.

5. Technologies Choices

The following technologies are selected to design and implement the RAG-based customer support assistant. Each technology plays a specific role in ensuring efficient data processing, retrieval, and response generation.

- **ChromaDB**

  ChromaDB is used as the vector database to store embeddings.

  It enables fast similarity search and efficient retrieval of relevant document chunks.

  It is lightweight and easy to integrate for small to medium-scale applications.

- **LangGraph**

  LangGraph is used to manage the workflow of the system.

  It allows defining nodes and edges to control the flow of execution.

  It helps implement decision-making logic and supports conditional routing.

- **LLM Choice (ChatGroq - LLaMA Model)**

  The system uses ChatGroq with the LLaMA model for generating responses.

  It provides fast and accurate answers based on the retrieved context.

  It is suitable for real-time customer support applications.

- **Additional Tools**

  LangChain is used for document loading, chunking, and retrieval.

  HuggingFace embeddings are used to convert text into vector representations.

  Python is used as the programming language for implementation.

6. Scalability Considerations

To ensure the system performs efficiently under different conditions, several scalability aspects are considered. These include handling large data, managing multiple user queries, and reducing response time.

- **Handling Large Documents**

  Large documents are handled by splitting them into smaller chunks.

  This ensures efficient processing and better retrieval accuracy.

- **Increasing Query Load**

  The system can handle multiple queries by scaling the vector database and using efficient retrieval mechanisms.

  Parallel processing can also be applied to improve performance.

- **Latency Concerns**

To reduce response time, optimized embeddings and fast LLMs are used.

Caching frequently asked queries can further improve system performance.